

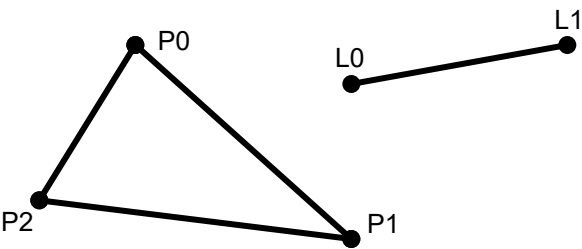
複雑なコリジョン3D

* 様々な3Dのコリジョンについてみてみよう。

クラス記号	出席番号	氏名
-------	------	----

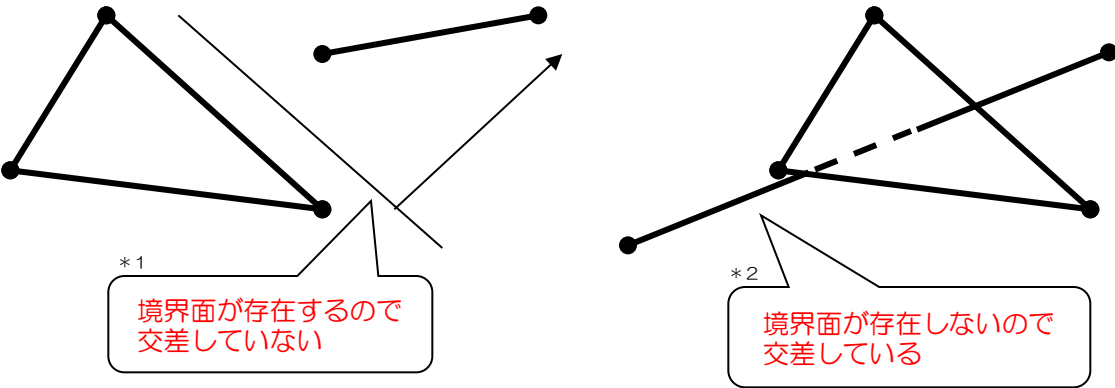
<<線分と三角形との交差判定>>

3Dゲームの中で、おそらく最も頻繁に利用されるであろうと思われる判定、三角形と線分との判定を扱ってみる。
三角形と線分との判定は、「弾が壁に衝突したかどうか」など多くのシーンで応用される。



線分の端点の座標
L0 (L0x, L0y, L0z)
L1 (L1x, L1y, L1z)
三角形の端点の座標
P0 (P0x, P0y, P0z)
P1 (P1x, P1y, P1z)
P2 (P2x, P2y, P2z)

(考え方)
線分と三角形との間に「境界面」が存在するかどうかで判定する。



このように2つの物体を隔てる存在である平面のことを「**分離平面**」と呼び、分離平面の「法線」のことを
*4「**分離軸**」と呼ぶ。

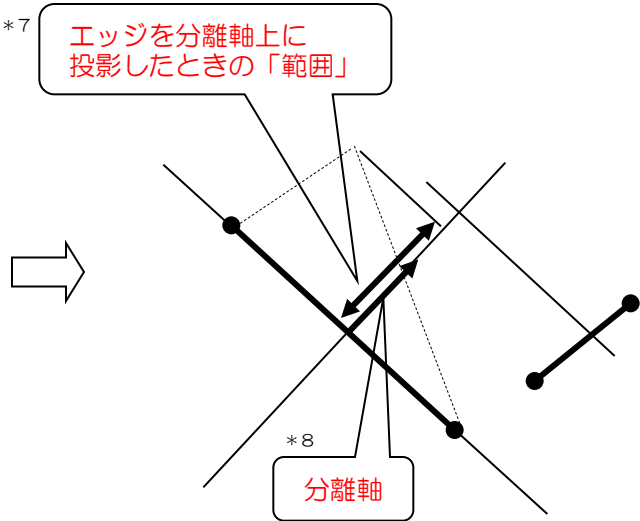
ここで、物体の衝突と分離軸との間には次のような関係がある。
*5

2つの物体が衝突していない場合、物体上の平面の法線あるいは2つの物体が持つ個々のエッジ両者に直交する軸のいずれかが分離軸となる。

さらに続けて考える。
*6

分離軸を含む直線上に判定に関係する座標を「投影」しておのおのの範囲が重なるように存在する場合は交差していると判断する。範囲が重ならない場合は交差していないと判断する。

(イメージ) 分離軸への投影とは、



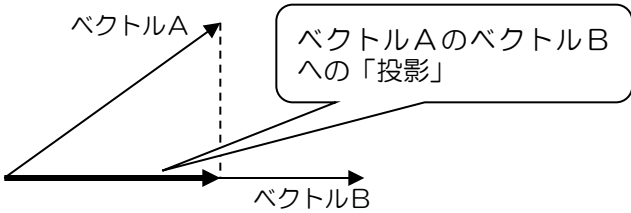
*9

分離軸チェックの対象が組み合わせにより多数発生する。
一度でも「交差していない」と判定できたときは物体同士は「衝突していない」と断定できる。全ての組み合わせで交差している場合はもちろん「衝突している」ことになる。

<<分離軸の判定>>

分離軸への投影で、範囲の重なりを判定する。

- ◎ 投影とは
ベクトルAをベクトルBへおろした時に生じるベクトルのことを指す。

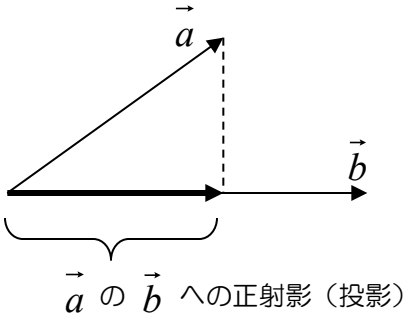


この投影ベクトルの大きさはベクトルBが単位ベクトルの場合、内積の値そのものになる。

(理由)
内積の定義より $\vec{a} \cdot \vec{b} = \|\vec{a}\| \cdot \|\vec{b}\| \cdot \cos \theta$

$\|\vec{a}\| \cdot \cos \theta$ は図からわかるように $\|\vec{b}\|$ が1のとき、 $\vec{a}$ の $\vec{b}$ への正射影の長さを意味している。

よって、 $\vec{b}$ が単位ベクトルなので内積を計算すると、その値が正射影の長さとなる。



<<実際の領域交差判定>>

三角形と線分を、パーツに分けて考える。

線分 $\cdots$ ^{*10} 1組のエッジが存在

三角形 $\cdots$ 1組の面と3組のエッジが存在

よって、交差判定を行う組み合わせは、次のようになる。

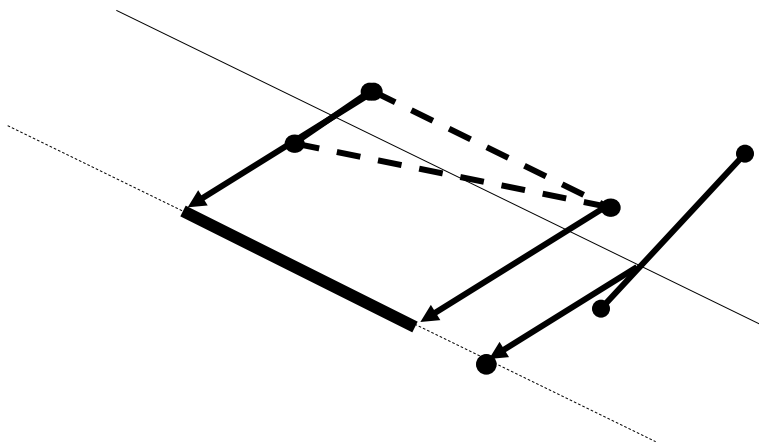
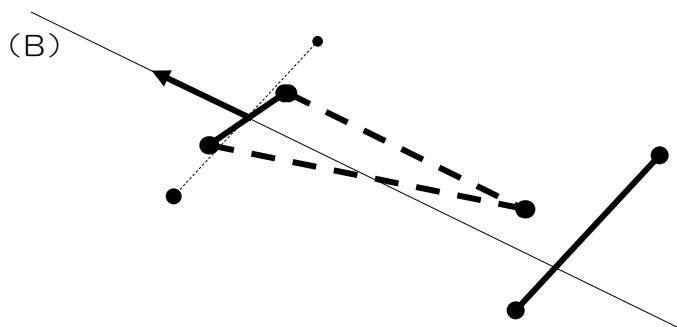
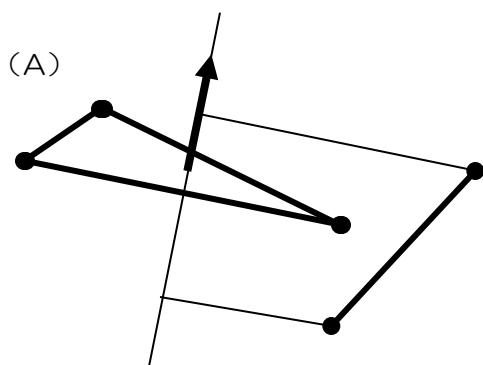
^{*11}

(A) 「三角形の面の法線」に対して

> 「線分の端点(2個)」と「三角形の頂点座標」を投影し、範囲判定

(B) 「三角形のエッジ(それぞれ)と線分との直交線に対して

> 「線分の端点」と「三角形の頂点座標(3つとも)」を投影し、範囲判定



具体的に考えるためベクトルを設定する。

線分LOL1 $\cdots$ ^{*12} $\overrightarrow{LOL1} = (L1_x - L0_x, L1_y - L0_y, L1_z - L0_z)$

三角形の頂点が P0, P1, P2 なので、分離軸の候補は以下の4本となる。

^{*13}

$$n1 = \overrightarrow{P0P1} \times \overrightarrow{P1P2} \quad \cdots \text{面の法線}$$

$$n2 = \overrightarrow{P0P1} \times \overrightarrow{LOL1} \quad \cdots \text{辺と線分との分離軸1}$$

$$n3 = \overrightarrow{P1P2} \times \overrightarrow{LOL1} \quad \cdots \text{辺と線分との分離軸2}$$

$$n4 = \overrightarrow{P2P0} \times \overrightarrow{LOL1} \quad \cdots \text{辺と線分との分離軸3}$$

◎ 次のパターンで、投影→判定を行う

判定処理

判定値

*14

$$(1) \quad n1 \cdot L0, \quad n1 \cdot L1$$

$$n1 \cdot P0$$

$$(2) \quad \underbrace{n2 \cdot P0, \quad n2 \cdot P1, \quad n2 \cdot P2}$$

$$n2 \cdot L0$$

$$(3) \quad n3 \cdot P0, \quad \underbrace{n3 \cdot P1, \quad n3 \cdot P2}$$

$$n3 \cdot L0$$

$$(4) \quad \underbrace{n4 \cdot P0, \quad n4 \cdot P1, \quad n4 \cdot P2}$$

$$n4 \cdot L0$$

(注意)
三角形の座標(3点)を投影して範囲を求めるときは、3点から「最大値、最小値」を求める必要がある。

(注意)

- 判定値についてはすべての頂点について調べる必要がない点に注意。
理由は、場合によってはすべての頂点が投影するベクトルに垂直な位置関係となり、投影したときに同じ結果となり重複してしまうためである。

◎ プログラムの流れは次のようになる。

*15

(Phase 1) *上記(1)

- 三角形の法線ベクトル(正規化済み)を求める
- 法線ベクトルに対して直線を投影した結果を求める(2点存在。内積を使う)
- 三角形の頂点(どれか一つ)が投影した範囲内かどうか調査する
- 範囲外なら分離面が存在するとして「衝突していない」とする

(Phase 2) *上記(2)、(3)、(4)

- 三角形のエッジを表すベクトルと線分に直交するベクトルを外積で求める(正規化も必要)
- 直交ベクトルに三角形の3頂点を投影する(内積を使う)
- 投影した3点から最小値、最大値を求めて「範囲」を求める
- 線分の頂点(どちらか一つ)が範囲内かどうか調査する
- 範囲外なら分離面が存在するとして「衝突していない」とする
- Phase2 の処理をエッジ3つ分について繰り返す